

Chapter 1

Deep Learning

This chapter introduces the fundamental concepts of Deep Learning (DL), providing the necessary foundation for the ideas developed throughout the rest of this manuscript. This chapter can be seen as a synthesis of the reference work Deep Learning [?]. Its structure is as follows:

Section 1.1: An overview of the historical developments that led to the emergence of the field.

Section 1.2: A formalization of the objective underlying DL algorithms.

Section 1.3: An examination of the structure of artificial Neural Network (NN).

Section 1.4: An explanation of how the learning process shapes NN.

Section 1.5: A discussion of how such models are evaluated.

1.1 Historical Developments

Figure 1.1 provides a timeline of the key milestones that shaped deep learning, from the earliest mathematical model of artificial neurons to the rise of modern transformer architectures.

DL and NN emerged from an ambition to study the expressive capabilities of organic brain. They can be seen as a particularly striking example of biomimicry, as they draw inspiration from living systems to address scientific challenges. The first major milestone came in 1943 with the proposal of a mathematical model of an artificial neuron, with a formulation of networked interactions [?]. This work established the key idea that networks of simple units can give rise to complex computations. The focus was on representational power, not on the ability of such systems to learn. As a result, this work remained largely theoretical and had limited practical applications. A major step toward operationalizing the idea of networks with the perceptron algorithm in 1958 [?]. The perceptron was designed to adjusting its parameters based on input-output pairs in order to approximate a function. However, the perceptron's fundamental limitations were exposed in 1969, revealing that the algorithm could not learn functions as

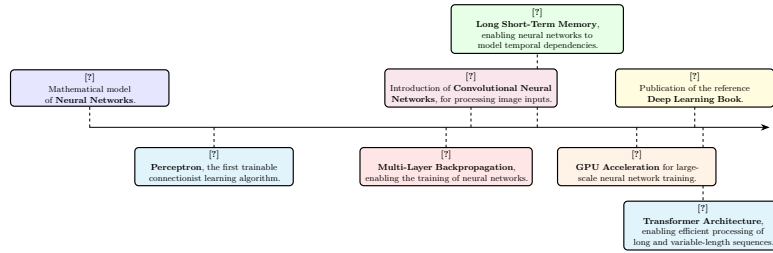


Figure 1.1: Evolution of deep learning.

simple as the exclusive-or [?]. To overcome this limitation, researchers in 1986 proposed using gradient descent optimization to train deep NNs [?].

It took several decades before DL reached the level of prominence it has today. One major obstacle was the absence of theoretical guarantees regarding the convergence of learning algorithms. Although the universal approximation theorem shows that NN can represent a wide class of functions [?], it did not ensure that such representations could be effectively learned through optimization methods like gradient descent. This lack of rigorous guarantees initially reduced interest within the academic community. As a result, early progress relied largely on empirical breakthroughs.

These empirical advances were made possible by two key developments: the availability of large-scale datasets and significant improvements in computational infrastructure. DL methods require vast amounts of data to perform well, and the rise of the internet alongside the digitization of information enabled the creation of such datasets [?]. At the same time, progress in hardware, particularly the use of graphics processing units, greatly increased computational capacity and made it feasible to train large-scale models [?].

These societal changes paved the way for the rise of DL as we know it today. DL has achieved remarkable success across a wide range of domains, including computer vision [?], complex games [?], content recommendation [?], bio-chemistry [?], as well as the now essential digital assistants based on large language models [?]. The diversity of these applications explains the enthusiasm for this technology, which continues to redefine the boundaries of what was once considered automatable.

1.2 Objective

Now that we have outlined the main developments that led to the rise of DL, we need to formalize what we are actually trying to achieve when performing DL. The range of problems that can be addressed with DL is extremely broad. To ensure clarity, we will consider the supervised learning setting, specifically applied to a regression task. This framework serves as a standard reference in the field, as all building blocks of DL applications are built upon this fundamental formalism.

In this setting, learning consists of progressively shaping a NN that serves as an approximation of a target function. At initialization, the network does not yet provide a meaningful representation of this function. It gradually becomes capable of capturing the relationship between inputs and outputs through

successive updates during training. For this reason, DL is naturally situated within the framework of machine learning. A standard definition of machine learning is:

“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .” [?]

When this formulation is adapted to DL, we obtain:

- The computer program corresponds to the learning algorithm, which is responsible for adjusting the network’s parameters.
- The task T consists of learning a NN that provides a good approximation of the target function.
- The performance measure P corresponds to a function used to evaluate the quality of this approximation.
- The experience E takes the form of a dataset composed of input–output pairs derived from the function to be approximated.

We now introduce the mathematical formalism used in this chapter in order to remove any ambiguity in the developments that follow. The function we aim to approximate is denoted by f . As with any function, it defines a mapping from an input to an output: we write $x \in \mathcal{X}$ for an input and $y \in \mathcal{Y}$ for the corresponding output. The function f is therefore characterized by an input space $\mathcal{X}$ and an output space $\mathcal{Y}$, and can be formally written as $f : \mathcal{X} \rightarrow \mathcal{Y}$. For simplicity of exposition and notation, we will assume in what follows that both $\mathcal{X}$ and $\mathcal{Y}$ are vector spaces, while keeping in mind that this framework can be readily generalized to more settings. Throughout this manuscript, the notation $\hat{\cdot}$ will be used to indicate that a given object is an approximation of another. Since the goal of the NN we train is to approximate f , we denote this approximation by $\hat{f} : \mathcal{X} \rightarrow \mathcal{Y}$. Given an input $x \in \mathcal{X}$, the output of the approximating function $\hat{f}(x)$ is denoted by $\hat{y}$.

It is important to emphasize that the target function f is generally not known explicitly. If it were, there would be no need to learn an approximation, since the true function would already provide the exact mapping. In practice, we only have access to a finite set of observations generated by f , organized in what is called a dataset. To distinguish between the different samples, we introduce an index i , allowing us to define input–output pairs of the form $(x^{(i)}, y^{(i)})$. This leads to the following formulation for a dataset $\mathcal{D}$ of size N , where N denotes the number of available examples:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N.$$

Now that we have made explicit how the target function f is represented, we can ask what it means for a function $\hat{f}$ to be a good approximation of this function. It means that, for a given input $x^{(i)}$, the NN output $\hat{y}^{(i)}$ is close to the true output $y^{(i)}$. To quantify the quality of this approximation, we introduce a function $\mathcal{L}$ that measures the discrepancy between predictions and target values. In DL, this function called the loss function plays a central role as it guides the

learning process. In regression settings, a common choice is the mean squared error, defined as:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2.$$

Thanks to this loss, it is now possible to rigorously define the performance measure P . This performance measure corresponds to the average loss over the dataset and can therefore be written as:

$$P = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\hat{f}(x^{(i)}), y^{(i)}).$$

This formulation captures the mathematical core of the learning objective. We seek a NN $\hat{f}$ that minimizes the prediction error over the dataset $\mathcal{D}$. As you can see, we are working in a very general setting, where the target function f may take many different forms. Consequently, the network $\hat{f}$ must be expressive enough to approximate a wide variety of such functions. This requirement motivates the choice of NN architectures, which are specifically designed to represent highly diverse functional mappings.

1.3 Neural Network Structure

NN are called connectionist models. Rather than modeling information processing through a set of explicit rules, they rely on a flow of signals distributed within a computational structure. Their architecture is composed of a large number of simple computational units, which operate on partial representations and exchange signals across the network. A complex pattern of weighted connections between these units enables the transformation and propagation of information.

To simplify the presentation throughout this chapter, we focus on the Multi-Layer Perceptron (MLP) architecture. Although more specialized architectures exist, the MLP forms a fundamental building block of modern DL. Indeed, most of the essential mechanisms of DL are present in it, making it a particularly suitable framework for introducing key concepts.

1.3.1 Neurons

The artificial neuron is the basic building block of NN. It is through these units that all computations within the network are carried out. A neuron receives a set of input signals from other neurons and produces an output that is transmitted to subsequent neurons units. Its internal state, called the activation, reflects its level of response and encodes the information processed for a given input. This activation, or lack thereof, is then used as input information by other neurons to determine their own activations, thereby propagating information through the network.

A neuron is characterized by:

- a set of incoming connections from other neurons,
- a weight associated with each of these connections,
- a bias term,
- a differentiable nonlinear activation function,

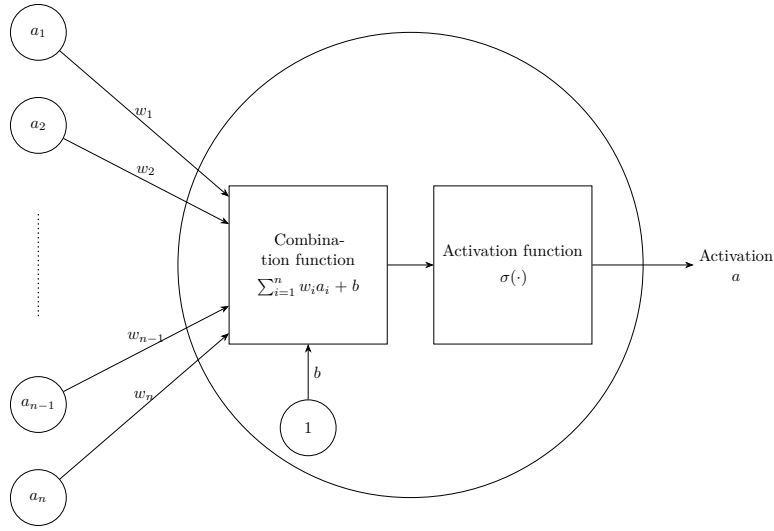


Figure 1.2: Artificiale neurone.

- a scalar activation state.

The activation of a neuron is determined by a weighted combination of its input signals, to which a bias is added, followed by the application of a nonlinear activation function. Mathematically, for a neuron receiving inputs $(a_1, \dots, a_n)$, its activation a is given by:

$$a = \sigma \left(\sum_{i=1}^n w_i a_i + b \right),$$

where:

- w_i is the weight associated with the i -th incoming connection,
- b is the bias,
- σ is the activation function.

Figure 1.2 shows a schematic view of this basic unit. The set of weights $w_1, \dots, w_i$ with the bias b constitute the parameters of the neuron, and they are the elements that evolve during training. The magnitude of a weight w (relative to the others) indicates how strongly a neuron depends on the activation of the neurons to which it is connected. A large positive weight means that a high activation in the connected neuron tends to increase the activation of the receiving neuron, whereas a negative weight implies the opposite effect. In this way, the weights modulate the strength and nature of the connections between neurons, thereby defining the overall pattern of interaction within the network.

The activation function σ models how a neuron transforms incoming signals into its output activation. Originally, this function was inspired by biological neurons, with the goal of mimicking the firing behavior of organic cells. However, modern DL no longer relies on this biological interpretation. In practice, any function that is nonlinear, differentiable, and computationally efficient can be

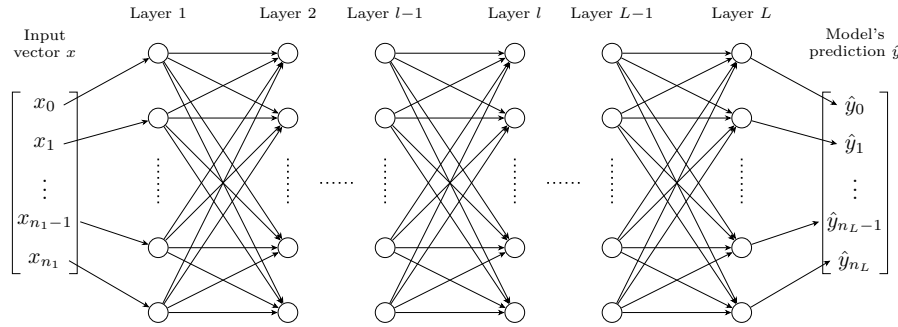


Figure 1.3: Multilayer Perceptron Architecture.

used. Nonlinearity is a crucial property, since without it a NN would reduce to a composition of linear transformations, and would therefore be unable to approximate complex functions.

1.3.2 Layers

In MLP architectures, neurons are organized into successive layers, as illustrated in Figure 1.3. This layered structure determines how neurons are connected. In this setting, each neuron receives inputs from all neurons in the preceding layer and sends its output to all neurons in the following layer. Consequently, neurons within the same layer do not interact with each other.

Three types of layers are distinguished:

- an input layer, which directly receives input,
- one or more hidden layers, which perform intermediate transformations,
- an output layer, which produces the final prediction.

The input and output layers play a specific role. The input layer has no preceding layer: its neurons do not compute activations but instead directly encode the components of the input vector x . Conversely, the output layer has no subsequent layer, and its neurons produce the model's prediction $\hat{y}$. This structure imposes a direct correspondence between the dimensionality of the input space and the number of neurons in the input layer, and similarly between the output space and the number of neurons in the output layer.

When describing NN, especially in MLP, it is more common to reason at the level of layers rather than individual neurons. This perspective allows interactions between layers to be modeled in a matrix form. Such a representation enables efficient parallel computation and forms the basis of modern DL implementations [?]. Within this framework, it is useful to introduce the notion of layer activations. The activation of a layer corresponds to the activations of all its neurons, organized into a vector. For a layer l composed of n_l neurons, the activations are given by:

$$\mathbf{h}^{(l)} = [a_1^{(l)}, \dots, a_{n_l}^{(l)}].$$

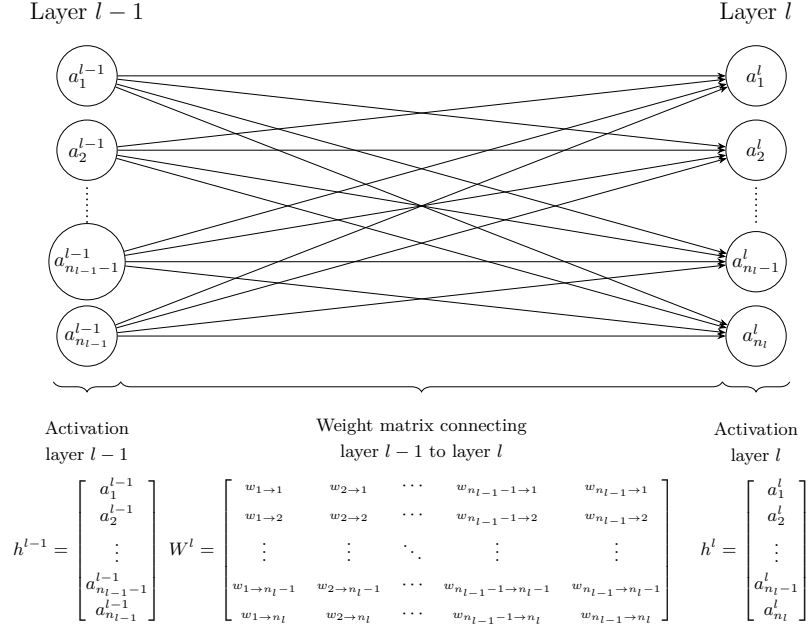


Figure 1.4: Interaction between layers.

Using this formulation, the interaction between two consecutive layers can be written compactly as:

$$\mathbf{h}^{(l)} = \sigma^{(l)} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right),$$

where:

- $\mathbf{h}^{(l-1)}$ represents the activations vector of the previous layer,
- $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ is the weight matrix connecting layer $l - 1$ to layer l ,
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ is the bias vector associated with layer l ,
- $\sigma^{(l)}$ is the activation function applied element-wise to the neurons of layer l .

This matrix-vector representation is visualised in Figure 1.4, which highlights how each neuron in layer l receives weighted signals from all neurons in the previous layer.

It is important to examine the role played by the intermediate layers of a NN. At least one hidden layer is required to represent sufficiently complex functions. In practice, however, modern architectures often include many more. This depth is mainly motivated by empirical observations. Each layer is seen as a processing step, and more complex target functions typically require more such steps to be well approximated. However, this interpretation is based on experimental findings rather than a theoretical justification [?]. An entire chapter of this manuscript is devoted to studying the information that can be extracted from the activations of these intermediate layers.

218 1.3.3 Network

219 With a layer-wise representation, the propagation of information can be described
 220 as a sequence of transformations applied from the input layer (indexed 1) to the
 221 output layer (indexed L):

$$\hat{f}(x) = \mathbf{h}^{(L)} = \sigma^{(L)} \left(\mathbf{W}^{(L)} \sigma^{(L-1)} \left(\dots \sigma^{(1)} \left(\mathbf{W}^{(1)} x + \mathbf{b}^{(1)} \right) \dots \right) + \mathbf{b}^{(L)} \right) = \hat{y}.$$

222 For a more compact description of information propagation, the layer activa-
 223 tions are defined recursively as:

$$\mathbf{h}^{(l)} = \begin{cases} x & \text{if } l = 1, \\ \sigma^{(l)} (\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) & \text{if } 2 \leq l \leq L. \end{cases}$$

224 To simplify notation, it is convenient to group all parameters of the network
 225 into a single set. The model parameters are thus denoted by:

$$\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^L.$$

226 In this manuscript, we consider θ as a vector in $\mathbb{R}^P$, where P denotes the
 227 total number of parameters $\theta = [w_1, \dots, w_P]$. The notation $\hat{f}_\theta$ expresses the
 228 dependence of the NN on its parameters. Increasing the number of parameters,
 229 either by adding more neurons per layer or by increasing the number of layers,
 230 enhances the expressive power of the model, allowing it to approximate more
 231 complex functions. However, this also increases computational cost and training
 232 time.

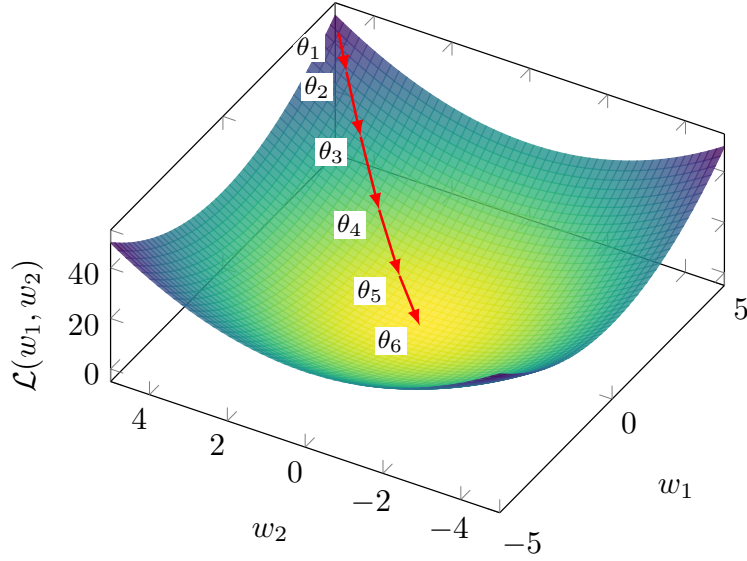
233 It is important to distinguish between the architecture and the parameters
 234 of a NN. When we refer to the architecture of $\hat{f}$, we mean the number of layers,
 235 the number of neurons per layer, and the choice of activation functions. The
 236 parameters correspond to θ , which controls the strength of the connections
 237 between neurons.

238 1.4 Learning

239 Now that we have detailed the structure of a NN, we must turn our attention to
 240 how it is modified in order to accomplish the task assigned to it. To do so, we
 241 need to focus on what lies at the very core of DL: the learning process itself. As
 242 a reminder, the objective of a NN is to adjust its parameters so that it becomes
 243 a good approximation of a target function. That is, for a given architecture $\hat{f}$,
 244 the goal is to find the set of parameters θ that best approximates the target
 245 function f . Mathematically, this objective is expressed as:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)}).$$

246 However, the NN $\hat{f}$ used to approximate the target function f is highly
 247 complex. It involves a large number of parameters as well as many nonlinear
 248 components. As a result, we cannot find an exact solution by directly studying
 249 the properties of the function. So, one must therefore rely on approximate
 250 methods to optimize NNs.

Figure 1.5: Gradient descent in a case $\mathcal{L}(w_1, w_2) = w_1^2 + w_2^2$.

The class of algorithms used for this optimization is known as gradient descent methods. To develop an intuition, Figure 1.5 illustrates gradient descent applied to the simple convex loss function $\mathcal{L}(w_1, w_2) = w_1^2 + w_2^2$, where the red path shows the parameter updates converging to the minimum.

For simplicity, we focus here on Stochastic Gradient Descent (SGD) with a batch size of 1. As in previous cases, this choice is made because this setting captures the essential intuition behind the training method.

The first step of SGD is to randomly initialize the parameter vector $\theta \in \mathbb{R}^P$ for a given network architecture $\hat{f}$. We denote this initial state by $\theta^{(1)}$. The goal is then to iteratively modify this vector in order to improve the network's ability to approximate the target function.

To determine how to update the parameters, we study the effect of small variations in each parameter on the loss $\mathcal{L}$. To analyze how parameter variations affect the loss, we rely on a fundamental mathematical tool known as the gradient. It is defined as the collection of partial derivatives of a function with respect to its variables. In our setting, the objective is to understand, for a given example $(x^{(i)}, y^{(i)})$, how the parameters should be adjusted so that the NN prediction $\hat{y}^{(i)}$ becomes closer to the true output $y^{(i)}$. To achieve this, we compute the partial derivatives of the loss function $\mathcal{L}$ with respect to each parameter vector θ :

$$\nabla_{\theta} \mathcal{L}(\hat{f}_{\theta}(x^{(i)}), y^{(i)}) = \left[\frac{\partial \mathcal{L}(\hat{f}_{\theta}(x^{(i)}), y^{(i)})}{\partial w_1}, \dots, \frac{\partial \mathcal{L}(\hat{f}_{\theta}(x^{(i)}), y^{(i)})}{\partial w_P} \right].$$

In neural network training, the loss surface is highly non-convex, meaning that the optimization path depends strongly on the initialisation. Figure 1.6 illustrates this behaviour for several well-known non-convex functions, showing how different starting points can lead to distinct local minima.

Since both the NN and the loss function are compositions of differentiable functions, these derivatives can be efficiently computed using the chain rule.

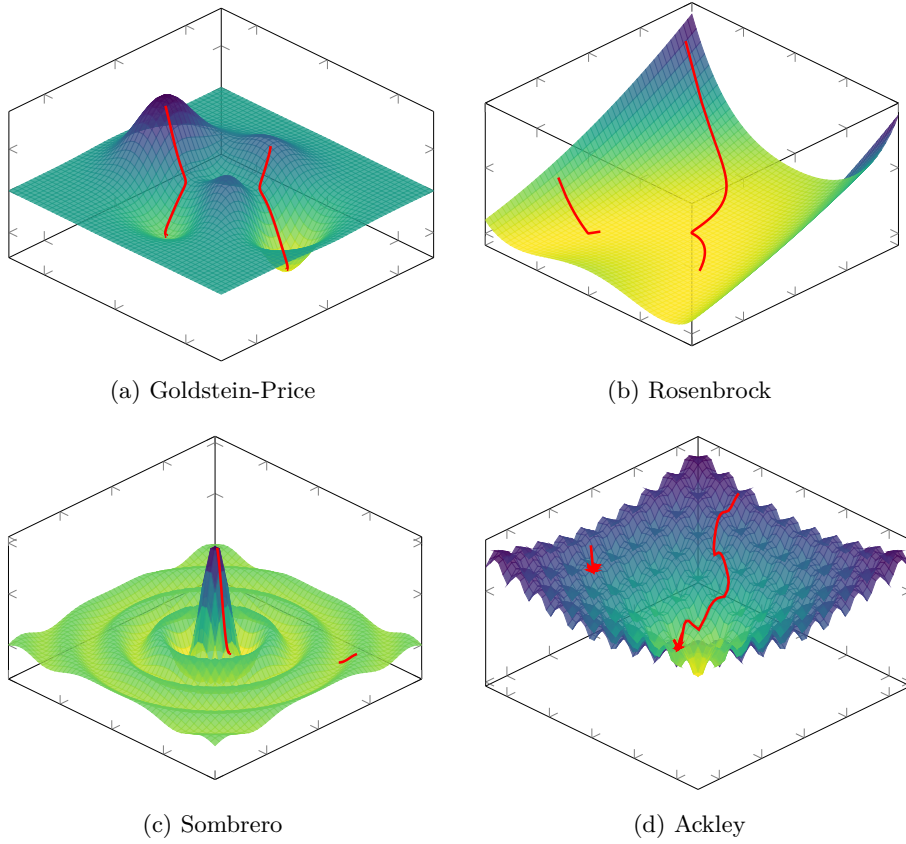


Figure 1.6: Comparison of loss surfaces and gradient descent trajectories.

Each component of the gradient $\frac{\partial \mathcal{L}(\dots)}{\partial w_k} \forall k \in [1, \dots, P]$ indicates how a small increase in the corresponding parameter w_k affects the loss. More precisely, for a given parameter, a positive value in the corresponding gradient component indicates that increasing this parameter will locally increase the loss. Conversely, a negative value indicates that increasing it will locally decrease the loss.

Now that we have information on how changes in the parameters affect the loss, we can aim to minimize it. To do so, the parameters are updated in the direction opposite to the gradient. Indeed, we move against the gradient because it indicates the direction in which the loss increases, whereas our goal is to reduce it. This leads us to the gradient descent update rule:

$$\theta^{(n+1)} = \theta^{(n)} - \eta \nabla_{\theta^{(n)}} \mathcal{L}(\hat{f}_{\theta^{(n)}}(x^{(i)}), y^{(i)}),$$

where $\eta > 0$ is the learning rate, which controls the step size. This parameter must remain small, as the gradient only provides reliable information in a local neighborhood of the current parameters. As a result, a single step is not sufficient to significantly reduce the loss, so the process is iterated many times. Producing a sequence of parameter vectors $\theta_1, \theta_2, \dots$ that progressively improve the model. In practice, training is stopped when successive updates no longer produce a significant decrease in the loss.

It is through this simple set of rules: random initialization, gradient computation, and iterative parameter updates, we are able to train highly complex NNs. Of course, when using gradient descent to optimize the parameters of a non-convex function such as a NN, there is no theoretical guarantee of convergence to a global, or even a local, minimum. However, in practice, this method often yields satisfactory results for the task. This success is often attributed to the high expressive power of NNs, even though a complete theoretical understanding of this phenomenon remains an open question.

1.5 Evaluation

Now that we have described how to train a model, we must turn to the question of how to evaluate it. In the supervised learning setting, this evaluation relies on two aspects:

- its performance on the data it was trained on,
- its performance on data it was not trained on.

The first aspect measures how well the loss minimization process has succeeded. The second measures how well the model generalizes. Indeed, the dataset $\mathcal{D}$ represents a small subset of all possible inputs. To estimate how the model will perform in general, it is important to evaluate how it performs on data it has not seen during training.

In order to quantify these two aspects, the dataset is split into two subsets:

- $\mathcal{D}_{\text{train}}$ is the set on which the model minimizes its loss during the training.
- $\mathcal{D}_{\text{test}}$ is kept aside during training in order to evaluate the model's ability to generalize.

When training is completed, the first step is to verify that the model performs well on the data it was trained on. To do so, we study the average loss on the training set, denoted $\bar{\ell}_{\text{train}}$:

$$\bar{\ell}_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{train}}} \mathcal{L}(\hat{f}_{\theta}(x^{(i)}), y^{(i)})$$

If $\bar{\ell}_{\text{train}}$ is not sufficiently low, this means that the model has not been able to learn the task. It is not necessary to go further in the analysis of the model's quality. Before restarting the training process, various design choices are then adjusted. Based on what we have discussed so far in this chapter, these include modifications to the number of layers, the number of neurons per layer, activation functions, the learning rate, or the initialization strategy. However, in practice, there are many other possible variations. There is no universal rule for choosing the appropriate value for each of them. They are therefore chosen empirically, based on models that perform well on similar tasks, combined with intuition. Their selection is often costly and remains a challenging problem in DL.

Assuming now that the model achieves satisfactory performance on the training dataset $\mathcal{D}_{\text{train}}$. We must now study its ability to generalize. To do so,

we compute the corresponding average loss on $\mathcal{D}_{\text{test}}$:

$$\bar{\ell}_{\text{test}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{test}}} \mathcal{L}(\hat{f}_{\theta}(x^{(i)}), y^{(i)})$$

If $\bar{\ell}_{\text{test}} \gg \bar{\ell}_{\text{train}}$, the model fails to generalize. This situation most often arises due to a phenomenon known as overfitting. In such cases, the model does not learn general patterns but instead memorizes the training examples. This typically occurs when the number of parameters is too large relative to the amount of training data. The simplest remedy is therefore to reduce the number of parameters in the model. However, it is still necessary to ensure that the model remains sufficiently expressive to perform well on the data.

If $\bar{\ell}_{\text{test}} \approx \bar{\ell}_{\text{train}}$, this indicates that the model generalizes well beyond the training data. The model can therefore be used with confidence for the task on which it was trained.

The ability of a neural network to generalize provides evidence that the model has learned more than a simple memorization of the training examples. Instead, it must construct internal representations that capture regularities present in the data. These representations emerge progressively through the hidden layers during training. Understanding their structure is therefore a key step toward understanding how neural networks generalize. The collection of these representations is referred to as the latent space. The next chapter is devoted to the study of these latent spaces.

Chapter 2

Latent Space Analysis

The ability of a neural network to generalize provides evidence that the model has learned more than a simple memorization of the training examples. Instead, it must construct internal representations that capture regularities present in the data. These representations emerge progressively through the hidden layers during training and play a central role in the network’s ability to solve a task. Understanding their structure is therefore a key step toward understanding how neural networks achieve generalization.

In classical machine learning approaches, practitioners had to manually design data representations in order to make them usable by simple models. Deep learning introduces a paradigm shift: rather than relying on hand-crafted features to meaningfully represent data, it automatically learns these representations. Through the successive activation of hidden layers, neural networks gradually construct representations that become increasingly relevant to the task under consideration. As discussed in Chapter 1, these representations are not meaningful at initialization, they emerge progressively during training as the model optimizes its objective. It is precisely through these learned representations that a trained network is able to map an input to an output, including for examples never encountered during training.

The study of these representations has therefore become a central topic in deep learning research. In particular, researchers seek to understand how information is organized within these representations, what concepts they encode, and how they contribute to the model’s behavior. The collection of representations produced by a hidden layer is referred as a latent space.

In this chapter, we first make explicit the working hypothesis that underlie most research on latent space analysis. To do so, we introduce the notions of abstraction, concept, latent space, and semantics. Once the notions of this conceptual framework has been established, we review the main methods that aim to identify and extract concepts from latent representations.

2.1 Hypothesis

In this section, we introduce the main working hypothesis commonly shared in the literature on latent space analysis. It should be noted that this hypothesis is rarely stated explicitly, but rather appears implicitly through the notions

of abstraction, concept, latent space and semantics, used throughout the field. Rather, it should be understood as a set of intuitions and assumptions guiding the study and interpretation of these spaces. The aim of this section is therefore to make this general conceptual framework explicit and to define the main terms associated with it.

This hypothesis aims to explain the generalization ability of neural networks observed after training has been completed, as seen in Section 1.5. Formally, generalization is defined as, even for an input $x \in X$ never seen during training, the network is often able to produce an output $\hat{y}$ close to the true output y . This ability implies that the network cannot simply memorize the associations observed during training. Instead, it must capture deeper regularities of the function it approximates. This process requires a form of abstraction: the network must be able to extract the relevant information contained in x and properly combine the elements in order to produce a coherent prediction.

This capacity for abstraction emerges through the construction of meaningful data representations during training. A neural network does not directly map an input x to an output y . Instead, as illustrated in Figure 1.3, multiple hidden layers lie between these two endpoints and produce intermediate activations. These activations constitute internal representations of the input. We refer to them as representations because each layer preserves certain properties of x while progressively reorganizing the information into a form better suited to the task. At each hidden layer, the network refines the representation of x , gradually transforming it into a form closer to the desired output. This process can therefore be viewed as a chain of successive transformations:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow \dots \rightarrow y$$

where each hidden state encodes a progressively more task-relevant representation of the input. These representations are considered abstract because they do not aim to preserve all details of the original input; instead, they retain only the most useful features for prediction.

If a network is able to construct sufficiently meaningful representations of data to ensure generalization, this implies that it can associate a unknown input with structures already encountered during training. These regularities are commonly referred to as concepts. A concept can be understood as a general and abstract representation of a situation, useful for accomplishing a given task. Although this notion remains difficult to formalize rigorously and is still relatively vague in the scientific literature, it intuitively refers to an abstract representation that groups together relevant information for problem-solving. The internal representations of a network can then be interpreted as a combination of several concepts relevant to prediction.

The set of activations of a given layer is referred to as the latent space. This space is not simply an unordered collection of numerical values. Rather, representations are organized according to a specific structure learned during training. The relative position of representations in this space directly influences the behavior of the network. Similar inputs generally produce nearby representations in the latent space, whereas different inputs occupy distinct regions. The latent space thus reflects both the properties of the data, the constraints of the task, and the internal mechanisms learned by the network. Studying these latent spaces is particularly important because it allows us to observe how the network structures information in order to make predictions.

This naturally leads to the question of whether the concepts learned by neural networks are similar to those used by humans when performing comparable tasks. When a human learns a task, they also build abstractions that allow them to generalize to new situations. Researchers therefore seek to determine whether the representational mechanisms observed in neural networks share similarities with those used in human intelligence. When a concept learned by a network corresponds to a human-interpretable concept, it is generally said to have semantic value. However, a network may also learn highly effective concepts for solving a task while remaining difficult for humans to interpret. In such cases, internal representations retain strong functional utility despite low semantic interpretability.

It is important to emphasize that the ideas presented in this section are not yet formally established. The mathematical tools currently available do not provide a complete explanation of the internal functioning of deep neural networks, nor a rigorous proof of the existence of concepts within learned representations. Nevertheless, this perspective currently constitutes the dominant working hypothesis in the community studying latent spaces. Many research works therefore aim to characterize internal representations, identify the concepts present in hidden activations, and better understand how these structures emerge during training.

2.2 Probing

The simplest question one can ask about an agent’s internal representations is whether it encodes a given human-chosen concept. To address this, probing methods aim to determine, within embeddings, whether the presence or absence of a selected concept is encoded. The underlying idea is that if it is possible to distinguish the presence or absence of a concept within embeddings, this strongly suggests that the neural network likely uses this concept to accomplish its task.

A probing analysis begins by identifying a set of interpretable concepts that the network could potentially use to accomplish its task. Each example in the dataset is then annotated to indicate the presence or absence of the studied concept. The data are subsequently projected into the latent space of the pretrained model. From these latent representations, a classifier is trained in a supervised manner to predict the presence of the considered concept. This classifier is referred to as a probe, since its role is to determine whether a concept is present or not. The role of this probe is therefore to assess whether the information associated with the concept is accessible from the network’s internal representations.

The classifier used to detect the presence or absence of a concept must remain intentionally simple. Indeed, if the probe has too much capacity, it may reconstruct the target information from complex correlations that do not necessarily correspond to a representation explicitly encoded in the latent space. In such a case, it becomes difficult to determine whether the concept is genuinely represented by the network or merely reconstructed through the expressive power of the classifier. For this reason, probing methods often rely on linear classifiers. A linear separation suggests that the information is easily accessible to the neural network and potentially usable during decision making.

If the classifier achieves performance significantly above chance level, this

479 suggests that the latent space contains exploitable information related to the
480 considered concept. However, the fact that a concept can be decoded from latent
481 representations does not necessarily imply that the model actually uses this
482 information to perform its task.

483 To ensure that representations play a causal role in the model’s output, it is
484 possible to perform interventions on the latent representations. An intervention
485 consists of modifying a latent representation in a controlled manner by activating
486 or deactivating a given concept, and then studying its impact on the model’s
487 output. If the resulting change in the model’s output is consistent with the
488 applied perturbation, this suggests that the representation had a causal role.

489 The main limitation of this approach is that the concepts under investigation
490 must be defined in advance before any probing analysis can be performed. This
491 introduces a strong bias in what can be studied. Moreover, annotating a dataset
492 for each concept is extremely time-consuming and requires significant human
493 effort. Interventions in latent space are also difficult to interpret, as it is not
494 always clear what constitutes a meaningful change in the network’s output under
495 a given perturbation.

496 This set of limitations makes probing a highly human-centric approach,
497 strongly dependent on the choice of investigated concepts and on the interpreta-
498 tion of interventions. To overcome this bias, other methods for studying latent
499 spaces have been developed, particularly those aiming to discover concepts in an
500 unsupervised manner.